V. Sarcar, *Java Design Patterns*
https://doi.org/10.1007/978-1-4842-7971-7_1

1. Understanding SOLID Principles

Vaskaran Sarcar[1]
(1) Garia, Kolkata, India

Java is a powerful object-oriented programming language, and it has many features. If we compare it with the old days, we must say that coding has become easier with the support of these powerful features. But the hard truth is that simply using these features in an application does not guarantee that you have used them in the right way. In any given requirement, it is vital to identify classes, objects, and how they communicate with each other. In addition, your application must be flexible and extendable to fulfill future enhancements. This is one of the primary aims of learning design patterns. This is often termed *experience reuse* because you earn benefits from other people's experiences as you go through their struggles and see how they solved those problems and adopted new behaviors in their systems. A pattern may not perfectly fit into your target application, but if you know the best practices in advance, you are more likely to make a better application.

Hopefully, you can guess that understanding different design patterns may not be very easy at the beginning. You need to know certain principles or guidelines before you implement a pattern in your code. These fundamental guidelines are not only common to all patterns, they are also useful to produce good-quality software.

In the previous editions of this book, I needed to assume that you knew at least some of them. Whenever I referred to these principles, either I explained them briefly or I gave you pointers to where you could learn more. This strategy was helpful to make the book slim.

Now you are holding the third edition of *Java Design Patterns* . Since you liked the previous edition of this book, I wanted to start with materials that can make your learning experience easier. So, I start with some fundamental guidelines that every developer should know before implementing a design pattern. Yes, I sincerely believe the previous line. Let's learn some of these guidelines.

Robert Cecil Martin is a famous name in the programming world. He is an American software engineer and best-selling author and is also known as "Uncle Bob." He promoted many principles. The following is a subset of them:

- **S**ingle Responsibility Principle (SRP)
- **O**pen/Closed Principle (OCP)
- **L**iskov Substitution Principle (LSP)
- **I**nterface Segregation Principle (ISP)
- **D**ependency Inversion Principle (DIP)

Taking the first letter of each principle, Michael Feathers introduced the SOLID acronym to remember these names easily.

Design principles are high-level guidelines that you can use to make better software. They are not bound to any particular computer language. So, if you understand these concepts using Java, you can use them with similar languages like C# or C++. To understand the thoughts of Robert C Martin about this, go to **https://sites.google.com/site/unclebobconsultingllc/getting-a-solid-start**, which says

> *The SOLID principles are not rules. They are not laws. They are not perfect truths. They are statements on the order of "An apple a day keeps the doctor away." This is a good principle, it is good advice, but it's not a pure truth, nor is it a rule.*
>
> *Uncle Bob*

In this chapter, you'll explore these principles in detail. In each case, I start with a program that compiles and runs successfully, but does not follow any specific design guidelines. In the analysis section, I'll discuss the possible drawbacks and try to find a better solution using these principles. This process can help you understand the importance of these design guidelines. I remind you again: examining these case studies will help you think better and make better applications, but they are not rules that you must follow in every context.

Single Responsibility Principle

A class acts like a container that can hold many things such as data, properties, or methods. If you put in too much data or methods that are not related to each other, you end up with a bulky class that can create problems in the future. Let's consider an example. Suppose you create a class with multiple methods that do different things. In such a case, even if you make a small change in one method, you need to retest the whole class again to ensure the workflow is correct. Thus, changes in one method can impact the other related method(s) in the class. This is why the single responsibility principle opposes this idea of putting multiple responsibilities in a class. It says that **a class should have only one reason to change.**

So, before you make a class, identify the responsibility or purpose of the class. If multiple members help you achieve a single purpose, it is ok to place all the members inside the class.

Point To Remember

When you follow the SRP, your code is smaller, cleaner, and less fragile. So how do you follow this principle? A simple answer is you can divide a big problem into smaller chunks based on different responsibilities and put each of these small parts into separate classes. The next question is, what do we mean by responsibility? In simple words, **responsibility is a reason for a change**. In his best-selling book *Clean Architecture* (Pearson, 2017), Robert C. Martin warns us not to confuse this principle with the principle that says a function should do one, and only one, thing. He also says that perhaps the best way to understand this principle is when you look at the symptoms of violating it.

I also believe the same, not only for this principle but for other principles as well. This is why in the upcoming discussion you'll see me first write a

program without following these principles. Then I'll show you a better program following these principles.

Initial Program

Demonstration 1 has an `Employee` class with three different methods. Here are the details:

- The `displayEmpDetail()` shows the employee's name and their working experience in years.
- The `generateEmpId()` method generates an employee id using string concatenation. The logic is simple: I concatenate the first word of the first name with a random number to form an employee ID. In the following demonstration, inside the `main()` method (the client code) I create two `Employee` instances and use these methods to display the relevant details.
- The `checkSeniority()` method evaluates whether an employee is a senior person. I assume that if the employee has 5+ years of experience, he is a senior employee; otherwise, he is a junior employee.

Demonstration 1

Here is the complete demonstration. When you download the source code from the Apress website, refer to package `jdp3e.solid_principles.without_srp` to get all parts of this program.

```
// Employee.java
import java.util.Random;
class Employee {
    public String firstName, lastName, empId;
    public double experienceInYears;
    public Employee(String firstName, String lastName,
                    double experience) {
        this.firstName = firstName;
        this.lastName = lastName;
        this.experienceInYears = experience;
    }
    public void displayEmpDetail(){
              System.out.println("The employee name: " +
                         lastName+","+firstName);
              System.out.println("This employee has " +
                         experienceInYears +"
                         years of experience.");
    }
    public String checkSeniority(double experienceInYears){
        return  experienceInYears > 5 ?"senior":"junior";
    }
    public String generateEmpId(String empFirstName){
        int random = new Random().nextInt(1000);
        empId = empFirstName.substring(0,1)+random;
        return empId;
    }
}
// Client.java
class Client {
   public static void main(String[] args) {
       System.out.println("*** A demo without SRP.***");
              Employee robin = new Employee("Robin", "Smith", 7.5);
              showEmpDetail(robin);
              System.out.println("\n*******\n");
```

```
                Employee kevin = new Employee("Kevin", "Proctor", 3.2);
                showEmpDetail(kevin);
    }


  private static void showEmpDetail(Employee emp) {
        emp.displayEmpDetail();
        System.out.println("The employee id: "+
                  emp.generateEmpId(emp.firstName));
        System.out.println("This employee is a " +
                  emp.checkSeniority(emp.experienceInYears)+
                    " employee.");
    }
}
```

Note For brevity, I do not include the common package name before each segment of the code. The same comment applies to all demonstrations in this book.

Output

Here is a sample output. Note that the employee ID can vary in your case because it generates a random number to get the employee ID.

```
*** A demo without SRP.***
The employee name: Smith,Robin
This employee has 7.5 years of experience.
The employee id: R446
This employee is a senior employee.
*******
The employee name: Proctor,Kevin
This employee has 3.2 years of experience.
The employee id: K822
This employee is a junior employee.
```

Analysis

What is the problem with this design? This answer is that I violate the SRP here. You can see that displaying an employee detail, generating an employee id, or checking a seniority level are all different activities. Since I put everything in a single class, I may face problems adopting new changes in the future. Here are some possible reasons:

- The top management can set a different criterion to decide a seniority level.
- They can also use a complex algorithm to generate the employee id.

In each case, I'll need to modify the `Employee` class. Now you understand that it is better to follow the SRP and separate these activities.

Better Program

In the following demonstration, I introduce two more classes. The `SeniorityChecker` class now contains the `checkSeniority()` method and the `EmployeeIdGenerator` class contains the `generateEmpId(...)` method to generate the employee id. As a result, in the future, if I need to change the program logic to determine the seniority level or use a new algorithm to generate an employee id, I can make the changes in the re-

spective classes. Other classes are untouched, so I do not need to retest those classes.

To improve the code readability and avoid clumsiness inside the `main()` method, I use the static method `showEmpDetail(...)`. This method calls the `displayEmpDetail()` method from `Employee`, the `generateEmpId()` method from `EmployeeIdGenerator`, and the `checkSeniority()` method from `SeniorityChecker`. You understand that this method was not necessary, but it makes the client code simple and easily understandable.

Demonstration 2

Here is the complete demonstration that follows SRP. When you download the source code from the Apress website, refer to package `jdp3e.solid_principles.srp` to get all parts of this program.

```
// Employee.java
class Employee {
    public String firstName, lastName, empId;
              public double experienceInYears;
              public Employee(String firstName, String lastName,
                          double experience) {
                    this.firstName = firstName;
                    this.lastName = lastName;
                    this.experienceInYears = experience;
              }
              public void displayEmpDetail(){
                   System.out.println("The employee name:
                              "+lastName+","+firstName);
                        System.out.println("This employee has "+
                 experienceInYears+" years of experience.");
                 }
}
// EmployeeIdGenerator.java
import java.util.Random;
class EmployeeIdGenerator {
    String empId;
         public String generateEmpId(String empFirstName) {
             int random = new Random().nextInt(1000);
                           empId = empFirstName.substring(0, 1) + random;
                          return empId;
            }
}
// SeniorityChecker.java
class SeniorityChecker {
    public String checkSeniority(double experienceInYears){
            return  experienceInYears > 5 ?"senior":"junior";
    }
}
// Client.java
class Client {
  public static void main(String[] args) {
    System.out.println("*** A demo that follows the SRP.***");
    Employee robin = new Employee("Robin", "Smith", 7.5);
    showEmpDetail(robin);
    System.out.println("\n*******\n");
    Employee kevin = new Employee("Kevin", "Proctor", 3.2);
    showEmpDetail(kevin);
  }
    private static void showEmpDetail(Employee emp) {
```

```
        // Display employee detail
        emp.displayEmpDetail();
        // Generate the ID
        EmployeeIdGenerator idGenerator = new
                                    EmployeeIdGenerator();
        String empId = idGenerator.generateEmpId(emp.firstName);
        System.out.println("The employee id: " + empId);
        // Check the seniority level
        SeniorityChecker seniorityChecker = new
                                        SeniorityChecker();
        System.out.println("This employee is a " +
        seniorityChecker.checkSeniority(emp.experienceInYears)
                                    + " employee.");
    }
}
```

Output

Here is the output. Notice that it is similar to the previous output, except the first line that says this program follows the SRP now. (As I said before, the employee ID can vary in your case).

```
*** A demo that follows the SRP ***
The employee name: Smith,Robin
This employee has 7.5 years of experience.
The employee id: R168
This employee is a senior employee.
*******
The employee name: Proctor,Kevin
This employee has 3.2 years of experience.
The employee id: K258
This employee is a junior employee.
```

Point To Note

Note that the SRP does not say that a class should have at most one method. Here the emphasis is on the single responsibility. There may be closely related methods that can help you to implement a responsibility. For example, if you have different methods to display the first name, the last name, and a full name, you can put these methods in the same class. These methods are closely related, and it makes sense to place all these display methods inside the same class.

In addition, you should not conclude that you should always separate responsibilities in every application that you make. You need to analyze the change's nature. It is because too many classes can make your application complex and thus difficult to maintain. But if you know this principle and think carefully before you implement a design, you are likely to avoid the mistakes discussed earlier.

Open/Closed Principle

According to Robert C. Martin, the Open/Closed Principle is the most important principle among all the principles of object-oriented design. In the book *Clean Architecture*, he says the following:

> *The Open-Closed Principle (OCP) was coined in 1988 by Bertrand Meyer. It says:* **A software artifact should be open for extension but closed for modification.**

In this section, I'll examine the OCP principle in detail using Java classes. Reading *Object-Oriented Software Construction (Second Edition)* by Bertrand Meyer, I found some important thoughts behind this principle. Here are some of them:

- Any modular decomposition technique must satisfy the Open-Closed Principle. Modules should be both open and closed.
- The contradiction between the two terms is only apparent as they correspond to goals of a different nature.
  - A module is said to be open if it is still available for extension. For example, it should be possible to expand its set of operations or add fields to its data structures.
  - A module is said to be closed if it is available for use by other modules. This assumes that the module has been given a well-defined, stable description (its interface in the sense of information hiding). At the implementation level, closure for a module also implies that you may compile it, perhaps store it in a library, and make it available for others (its clients) to use.
- The need for modules to be closed and the need for them to remain open arise for different reasons.
- He explains that openness is useful for software developers because they can't foresee all the elements that a module may need in the future. But the "closed" modules will satisfy project managers because they want to complete the project instead of waiting for everyone to complete their parts.

The previous points are self-explanatory. You understand that the idea behind this design philosophy is that in a stable and working application, once you create a class and other parts of your application start using it, any further change in the class can cause the working application to break. If you require new features (or functionalities), instead of changing the existing class, you can extend it to adopt the new requirements. What is the benefit? Since you do not change the old code, your existing functionalities continue to work without any problems, and you can avoid testing them again. Instead, you test the "extended" part (or functionalities) only.

In 1988, Bertrand Meyer suggested the use of inheritance in this context. Wikipedia (**https://en.wikipedia.org/wiki/Open%E2%80%93closed_principle**) mentions his quote as follows:

> *"A class is closed, since it may be compiled, stored in a library, baselined, and used by client classes. But it is also open, since any new class may use it as a parent, adding new features. When a descendant class is defined, there is no need to change the original or to disturb its clients.*

But inheritance promotes tight coupling. In programming, we like to remove these tight couplings. Robert C. Martin improved the definition and made it polymorphic OCP. The new proposal uses abstract base classes that use the protocols instead of a superclass to allow different implementations. These protocols are closed for modification, and they provide another level of abstraction that enables loose coupling. In this chapter, we follow Robert C. Martin's idea that promotes polymorphic OCP.

Initial Program

Assume that there is a small group of students who take a semester examination. (To demonstrate this, I choose a small number of participants to

help you focus on the principle, not unnecessary details.) `Sam, Bob, John,` and `Kate` are the four students in this example. They all belong to the `Student` class. To make a `Student` class instance, you supply a name, registration number, and the marks obtained in the examination. You also mention whether a student belongs to the Science department or the Arts department.

For simplicity, let's assume a particular college has the following four departments:

- Computer Science
- Physics
- History
- English

So, you see the following lines of code in the upcoming example:

```
Student sam = new Student("Sam", "R1", 81.5, "Comp.Sc.");
Student bob = new Student("Bob", "R2", 72, "Physics");
Student john = new Student("John", "R3", 71, "History");
Student kate = new Student("Kate", "R4", 66.5, "English");
```

When a student opts for computer science or physics, we say that they opt for the science stream. Similarly, when a student belongs to the History or English department, they are an arts student.

Start with two instance methods in this example. The `displayResult()` displays the result with all necessary details of a student and the `evaluateDistinction()` method evaluates whether a student is eligible for a distinction certificate. If a science student scores above 80 in this examination, they get the certificate with distinction. But the criterion for an arts student is slightly relaxed. An arts student gets the distinction if their score is above 70.

I hope you understand the SRP that I discussed earlier. If it is the case, you won't want to place `displayResult()` and `evaluateDistinction()` in the same class, like the following:

```
class Student {
   // Some fields, if any
   public void displayResult() {
     // some code
   }
   public void evaluateDistinction() {
     // some code
   }
 // Some other code, if any
}
```

What are the problems in this code segment?

- First, it violates the SRP because both the `displayResult()` and the `evaluateDistinction()` methods are inside the `Student` class. These two methods are unrelated.
- In the future, the examining authority can change the distinction criteria. In this case, you'll need to change the `evaluateDistinction()` method . Does it solve the problem? In the current situation, the answer is yes. But a college authority can change

the distinction criteria again. How many times will you retest the `Student` class due to the modification of the `evaluateDistinction()` method?

- Remember that each time you modify the method, you change the containing class and you need to modify the existing test cases too.

You can see that every time the distinction criteria changes, you need to modify the `evaluateDistinction()` method in the `Student` class. **So, this class does not follow the SRP and it is also not closed for modification.**

Once you understand these problems, you can start with a better design that follows the SRP. Here are the main characteristics of the design:

- In the following program, `Student` and `DistinctionDecider` are two different classes.
- The `DistinctionDecider` class contains the `evaluateDistinction()` method in this example.
- To show the details of a student you can override the `toString()` method, instead of using the separate method `displayResult()`. So, inside the `Student` class, you see the `toString()` method now.
- Inside main(), you see the following line:

```
List<Student> enrolledStudents = enrollStudents();
```

- The `enrollStudents()` method creates a list of students. You use this list to print student details one by one. You also use the same list before you invoke `evaluateDistinction()` to identify the students who received the distinction.

Demonstration 3

Here is the complete demonstration. All parts of the program are inside the package `jdp3e.solid_principles.without_ocp`. Refer to this package when you download the source code from the Apress website.

```
// Student.java
class Student {
     String name;
     String regNumber;
     String department;
     double score;
     public Student(String name, String regNumber,
                    double score, String dept) {
          this.name = name;
          this.regNumber = regNumber;
          this.score = score;
          this.department = dept;
     }
     @Override
     public String toString() {
          return ("Name: " + name +
                  "\nReg Number: " + regNumber +
                  "\nDept:" + department +
                  "\nMarks:" + score +
                  "\n*******");
     }
}
// DistinctionDecider.java
```

```
import java.util.Arrays;
import java.util.List;
class DistinctionDecider {
List<String> science = Arrays.asList("Comp.Sc.","Physics");
List<String> arts = Arrays.asList("History","English");
 public void evaluateDistinction(Student student) {
   if (science.contains(student.department)) {
    if (student.score > 80) {
     System.out.println(student.regNumber+" has received
                        a distinction in science.");
     }
   }
   if (arts.contains(student.department)) {
    if (student.score > 70) {
     System.out.println(student.regNumber+" has received a
                        distinction in arts.");
               }
          }
     }
}
// Client.java
import java.util.ArrayList;
import java.util.List;
class Client {
   public static void main(String[] args) {
        System.out.println("*** A demo without OCP.***");
        List<Student> enrolledStudents = enrollStudents();
        // Display all results.
        System.out.println("===Results:===");
        for(Student student:enrolledStudents){
             System.out.println(student);
        }
        System.out.println("===Distinctions:===");
        DistinctionDecider distinctionDecider = new
                                        DistinctionDecider();
        // Evaluate distinctions.
        for(Student student:enrolledStudents){
              distinctionDecider.evaluateDistinction(student);
        }
 }
 private static List<Student> enrollStudents() {
    Student sam = new Student("Sam", "R1", 81.5, "Comp.Sc.");
    Student bob = new Student("Bob", "R2", 72, "Physics");
    Student john = new Student("John", "R3", 71, "History");
    Student kate = new Student("Kate", "R4", 66.5, "English");
    List<Student> students = new ArrayList<Student>();
    students.add(sam);
    students.add(bob);
    students.add(john);
    students.add(kate);
    return students;
   }
}
```

Output

When you run this program, you'll see the following output:

```
*** A demo without OCP.***
===Results:===
Name: Sam
Reg Number: R1
```

```
Dept:Comp.Sc.
Marks:81.5
*******
Name: Bob
Reg Number: R2
Dept:Physics
Marks:72.0
*******
Name: John
Reg Number: R3
Dept:History
Marks:71.0
*******
Name: Kate
Reg Number: R4
Dept:English
Marks:66.5
*******
===Distinctions:===
R1 has received a distinction in science.
R3 has received a distinction in arts.
```

Analysis

Now you are following the SRP. If in the future the examining authority changes the distinction criteria, you do not touch the `Student` class. So, this part is closed for modification. This solves one part of the problem. Now think about another future possibility:

- The college authority can introduce a new stream such as commerce and set a new distinction criterion for this stream.

You need to make some obvious changes again. For example, you need to modify the `evaluateDistinction()` method and add another `if` statement to consider commerce students. Now the question is, is it ok to modify the `evaluateDistinction()` method in this manner? Remember that each time you modify the method, you need to test the entire code workflow again.

You understand the problem now. In this demonstration, every time the distinction criteria changes, you need to modify the `evaluateDistinction()` method in the `DistinctionDecider` class. **So, this class is not closed for modification.**

Better Program

To tackle this problem, you can write a better program. The following program shows such an example. It's written following the OCP principle that suggests we **write code segments (such as classes, or methods) that are open for extension but closed for modification .**

> The OCP can be achieved in different ways, but abstraction is the heart of this principle. If you can design your application following the OCP, your application is flexible and extensible. It is not always easy to fully implement this principle, but partial OCP compliance can generate greater benefit to you. Also notice that you started demonstration 3 following the SRP. If you do not follow the OCP, you may end up with a class that performs multiple tasks, which means the SRP is broken.

For the current situation, you could leave the `Student` class as it is. But you want to improve the code. You understand that in the future you may need to consider a different stream, such as commerce. How do you choose a stream? It is based on the subject chosen by a student, right? So, in the upcoming example, you make the `Student` class abstract. `ArtsStudent` and `ScienceStudent` are the concrete classes that extend the `Subject` class and are used to provide the "department" information (in other words, the subject taken by a student). The following code shows a sample implementation for your ready reference:

```
abstract class Student {
     String name;
     String regNumber;
     double score;
     String department;
     public Student(String name,
                    String regNumber,
                    double score) {
          this.name = name;
          this.regNumber = regNumber;
          this.score = score;
     }
     public String toString() {
          return ("Name: " + name +
                  "\nReg Number: " + regNumber +
                  "\nDept:" + department +
                  "\nMarks:"     + score +
                  "\n*******");
     }
}
public class ArtsStudent extends Student{
    public ArtsStudent(String name,
                       String regNumber,
                       double score,
                       String dept) {
          super(name, regNumber, score);
          this.department = dept;
     }
}
// The ScienceStudent class is not shown here
```

The previous construct helps you enroll science students and arts students separately inside the client code as follows:

```
private static List<Student> enrollScienceStudents() {
 Student sam = new ScienceStudent("Sam","R1",81.5,"Comp.Sc.");
 Student bob = new ScienceStudent("Bob","R2",72,"Physics");
 List<Student> scienceStudents = new ArrayList<Student>();
 scienceStudents.add(sam);
 scienceStudents.add(bob);
 return scienceStudents;
}
private static List<Student> enrollArtsStudents() {
 Student john = new ArtsStudent("John", "R3", 71,"History");
 Student kate = new ArtsStudent("Kate", "R4", 66.5,"English");
 List<Student> artsStudents = new ArrayList<Student>();
 artsStudents.add(john);
 artsStudents.add(kate);
 return artsStudents;
}
```

Now let's focus on the most important changes. You need to tackle the evaluation method for distinction in a better way. So, you create the interface

`DistinctionDecider` that contains a method called `evaluateDistinction`. Here is the interface:

```
interface DistinctionDecider {
     void evaluateDistinction(Student student);
}
```

`ArtsDistinctionDecider` and `ScienceDistinctionDecider` implement this interface and override the `evaluateDistinction(...)` method to specify the evaluation criteria as per their need. In this way, the stream-specific distinction criteria is wrapped in an independent unit. Here is the code segment for you. The different criteria for each class are shown in bold.

```
// ScienceDistinctionDecider.java
class ScienceDistinctionDecider implements DistinctionDecider{
     @Override
     public void evaluateDistinction(Student student) {
          if (student.score > 80) {
               System.out.println(student.regNumber+" has
                         received a distinction in science.");
          }
     }
}
// ArtsDistinctionDecider.java
class ArtsDistinctionDecider implements DistinctionDecider{
     @Override
     public void evaluateDistinction(Student student) {
          if (student.score > 70) {
               System.out.println(student.regNumber+" has
                           received a distinction in arts.");
          }
     }
}
```

Note The `evaluateDistinction(...)` method accepts a `Student` parameter. It means now you can also pass an `ArtsStudent` object or a `ScienceStudent` object to this method.

The remaining code is easy, and you should not have any trouble understanding the following demonstration now.

Demonstration 4

Here is the modified program. All parts of the program are inside the package `jdp3e.solid_principles.ocp`.

```
// Student.java
abstract class Student {
     String name;
     String regNumber;
     double score;
     String department;
     public Student(String name,
                    String regNumber,
                    double score) {
          this.name = name;
          this.regNumber = regNumber;
          this.score = score;
     }
     @Override
      public String toString() {
          return ("Name: " + name +
```

```
                  "\nReg Number: " + regNumber +
                  "\nDept:" + department +
                  "\nMarks:"+ score +
                  "\n*******");
     }
}
// ArtsStudent.java
public class ArtsStudent extends Student{
     public ArtsStudent(String name,
                        String regNumber,
                        double score,
                        String dept) {
          super(name, regNumber, score);
          this.department = dept;
     }
}
// ScienceStudent.java
class ScienceStudent extends Student{
     public ScienceStudent(String name,
                           String regNumber,
                           double score,
                           String dept) {
          super(name, regNumber, score);
          this.department = dept;
     }
}
// DistinctionDecider.java
interface DistinctionDecider {
     void evaluateDistinction(Student student);
}
// ScienceDistinctionDecider.java
class ScienceDistinctionDecider implements DistinctionDecider{
     @Override
     public void evaluateDistinction(Student student) {
          if (student.score > 80) {
               System.out.println(student.regNumber+" has
                         received a distinction in science.");
          }
     }
}
// ArtsDistinctionDecider.java
class ArtsDistinctionDecider implements DistinctionDecider{
     @Override
     public void evaluateDistinction(Student student) {
          if (student.score > 70) {
               System.out.println(student.regNumber+" has
                           received a distinction in arts.");
          }
     }
}
// Client.java
import java.util.ArrayList;
import java.util.List;
class Client {
public static void main(String[] args) {
  System.out.println("*** A demo that follows
                                  the OCP.***");
   List<Student> scienceStudents = enrollScienceStudents();
   List<Student> artsStudents = enrollArtsStudents();
      // Display all results.
     System.out.println("===Results:===");
     for (Student student : scienceStudents) {
          System.out.println(student);
     }
```

```
     for (Student student : artsStudents) {
          System.out.println(student);
     }
     // Evaluate distinctions.
     DistinctionDecider scienceDistinctionDecider =
                        new ScienceDistinctionDecider();
     DistinctionDecider artsDistinctionDecider =
                        new ArtsDistinctionDecider();
     System.out.println("===Distinctions:===");
     for (Student student : scienceStudents) {
      scienceDistinctionDecider.evaluateDistinction(student);
     }
     for (Student student : artsStudents) {
      artsDistinctionDecider.evaluateDistinction(student);
     }
   }
private static List<Student> enrollScienceStudents() {
 Student sam = new ScienceStudent("Sam","R1",81.5,"Comp.Sc.");
 Student bob = new ScienceStudent("Bob","R2",72,"Physics");
 List<Student> scienceStudents = new ArrayList<Student>();
 scienceStudents.add(sam);
 scienceStudents.add(bob);
 return scienceStudents;
}
private static List<Student> enrollArtsStudents() {
 Student john = new ArtsStudent("John", "R3", 71,"History");
 Student kate = new ArtsStudent("Kate", "R4", 66.5,"English");
 List<Student> artsStudents = new ArrayList<Student>();
 artsStudents.add(john);
 artsStudents.add(kate);
 return artsStudents


;
 }
}
```

Output

Notice that the output is the same except for the first line that says this program follows the OCP.

```
*** A demo that follows the OCP.***
===Results:===
Name: Sam
Reg Number: R1
Dept:Comp.Sc.
Marks:81.5
*******
Name: Bob
Reg Number: R2
Dept:Physics
Marks:72.0
*******
Name: John
Reg Number: R3
Dept:History
Marks:71.0
*******
Name: Kate
Reg Number: R4
Dept:English
```

```
Marks:66.5
*******
===Distinctions:===
R1 has received a distinction in science.
R3 has received a distinction in arts.
```

Analysis

What are the key advantages now? The following points tell you the answers:

- The `Student` class and `DistinctionDecider` are both unchanged for any future changes in the distinction criteria. They are closed for modification.
- Notice that every participant follows the SRP.
- Suppose you need to consider a new stream, say commerce. Then you can create a new class such as `CommerceStudent`. Notice that in a case like this, you do not need to touch the `ArtsStudent` or `ScienceStudent` classes.
- Similarly, when you consider different evaluation criteria for a different stream such as commerce, you can add a new derived class such as `CommerceDistinctionDecider` that implements the `DistinctionDecider` interface and you can set new distinction criteria for commerce students. In this case, you do not need to alter any existing class in the `DistinctionDecider` hierarchy. Obviously, the client code needs to adopt this change.
- Using this approach, you avoid an `if-else` chain (shown in demonstration 3). This chain could grow if you consider new streams such as commerce following the approach that is shown in demonstration 3. Remember that avoiding a big `if-else` chain is always considered a better practice. It is because avoiding the `if-else` chains lowers the cyclomatic complexity of a program and produces better code. (Cyclomatic complexity is a software metric to indicate the complexity of a program. It indicates the number of paths through a particular piece of code. So, in simple terms, by lowering the cyclomatic complexity you make your code easily readable and testable.)

I'll finish this section with Robert C. Martin's suggestion. In his book *Clean Architecture*, he gave us a simple formula: if you want component A to protect from component B, component B should depend on component A. But why do we give component A such importance? It is because we may want to put the most important rules in it.

It is time to study the next principle.

Liskov Substitution Principle

The Liskov Substitution Principle was initially introduced from the work of Barbara Liskov in 1988. **The LSP says that you should be able to substitute a parent (or base) type with a subtype**. It means that in a program segment, you can use a derived class instead of its base class without altering the correctness of the program.

How do you use inheritance? There is a base class and you create one (or more) derived classes from it. Then you can add new methods in the derived classes. As long as you directly use the derived class method with a

derived class object, everything is fine. A problem may occur if you try to get the polymorphic behavior without following the LSP. How? You'll see a detailed discussion with examples in this chapter.

Let me give you a brief idea. Assume that there are two classes in which B is the base class and D is the subclass (of B). Furthermore, assume that there is a method that accepts a reference of B as an argument, something like the following:

```
public void someMethod(B b){
       // Some code
  }
```

This method works fine until you pass a B instance to it. But what happens if you pass a D instance instead of a B instance? Ideally, the program should not fail. It is because you use the concept of polymorphism and you say D is basically a B type since class D inherits from class B. You can relate this scenario with a common example when we say a soccer player is also a player, where we consider the "player" class is a supertype of "soccer player."

Now see what the LSP suggests to us? It says that `someMethod()` should not misbehave/fail if you pass a D instance instead of a B instance to it. But it may happen if you do not write your code following the LSP. The concept will be clearer to you when you go through the upcoming example.

Point To Note
In design patterns, you often see polymorphic code. Here is a common example. Suppose you have the following code segment:

```
class B{}
class D extends B{}
```

Now you can write `B obB=new B();` for sure. But notice that in this case, you can also write

```
B obB=new D(); // Also Ok
```

Similarly, you can use the interfaces as the supertype. For example, if you have

```
interface B{}
class D implements B{}
```

you can write

```
B obB=new D(); // Also Ok
```

Polymorphic code shows your expertise but remember that it's your responsibility to implement polymorphic behavior properly and avoid unwanted outcomes.

Initial Program

Consider an example I see every month. I use an online payment portal to pay my electricity bill. Since I am a registered user, when I raise a pay-

ment request in this portal, it shows my previous payments too. Let's consider a simplified example based on this real-life scenario.

Assume that you also have a payment portal where a registered user can raise a payment request. You use method `newPayment()` for this. In this portal, you can also show the user's last payment details using a method called `previousPaymentInfo()`. Here is a sample code segment for this:

```
interface Payment {
     void previousPaymentInfo();
     void newPayment();
}
class RegisteredUserPayment implements Payment {
    String name;
    public RegisteredUserPayment(String userName) {
        this.name = userName;
    }
    @Override
    public void previousPaymentInfo(){
        System.out.println("Retrieving "+ name+"'s last
                           payment details.");
        // Some other code,if any
    }
    @Override
    public void newPayment(){
         System.out.println("Processing "+name+"'s current payment
                           request.");
         // Some other code, if any
    }
}
```

Furthermore, you create the helper class `PaymentHelper` to display all the previous payments and the new payment requests of these users. You use `showPreviousPayments()` and `processNewPayments()` for these activities. These methods call `previousPaymentInfo()` and `newPayment()` on the respective `Payment` instances. You use an enhanced `for` statement (it's often referred to as an enhanced `for` loop) to serve these purposes. Here is the `PaymentHelper` class for your instant reference:

```
import java.util.ArrayList;
import java.util.List;
public class PaymentHelper {
    List<Payment> payments = new ArrayList<Payment>();
    public void addUser(Payment user){
         payments.add(user);
    }
    public void showPreviousPayments() {
        for (Payment payment: payments) {
             payment.previousPaymentInfo();
             System.out.println("------");
        }
    }
    public void processNewPayments()  {
            for (Payment payment: payments) {
           payment.newPayment();
           System.out.println("------");
        }
    }
}
```

Inside the client code, you create two users and show their current payment requests along with previous payments. Everything is ok so far.

Demonstration 5

Here is the complete demonstration. All parts of the program are inside the package `jdp3e.solid_principles.without_lsp`. Refer to this package when you download the source code from the Apress website.

```
// Payment.java
interface Payment {
     void previousPaymentInfo();
     void newPayment();
}
// RegisteredUserPayment.java
class RegisteredUserPayment implements Payment {
    String name;
    public RegisteredUserPayment(String userName) {
      this.name = userName;
    }
    @Override
    public void previousPaymentInfo(){
      System.out.println("Retrieving "+ name+"'s last payment details.");
        // Some other code, if any
    }
    @Override
    public void newPayment(){
         System.out.println("Processing "+name+"'s current payment
                             request.");
         // Some other code, if any
    }
}
// PaymentHelper.java
import java.util.ArrayList;
import java.util.List;
public class PaymentHelper {
    List<Payment> payments = new ArrayList<Payment>();
    public void addUser(Payment user){
         payments.add(user);
    }
    public void showPreviousPayments() {
        for (Payment payment: payments) {
             payment.previousPaymentInfo();
             System.out.println("------");
        }
    }
    public void processNewPayments()  {
            for (Payment payment: payments) {
              payment.newPayment();
              System.out.println("------");
        }
    }
}
// Client.java
class Client {
     public static void main(String[] args) {
       System.out.println("***A demo without LSP.***\n");
        PaymentHelper helper = new PaymentHelper();
        // Instantiating two registered users
        RegisteredUserPayment robinPayment = new
                            RegisteredUserPayment("Robin");
        RegisteredUserPayment jackPayment = new
                            RegisteredUserPayment("Jack");
```

```
        // Adding the users to the helper
        helper.addUser(robinPayment);
        helper.addUser(jackPayment);
        // Processing the payments using
        // the helper class.
        helper.showPreviousPayments();
        helper.processNewPayments();
     }
}
```

Output

When you run this program, you'll see the following output:

```
***A demo without LSP.***
Retrieving Robin's last payment details.
------
Retrieving Jack's last payment details.
------
Processing Robin's current payment request.
------
Processing Jack's current payment request.
------
```

This program seems to be fine. **Now assume that you have a new requirement that says you need to support guest users in the future.** You can process a guest user's payment request, but you do not show their last payment detail. So, you create the following class that implements the `Payment` interface as follows:

```
class GuestUserPayment implements Payment {
    String name;
    public GuestUserPayment() {
        this.name = "guest";
    }
    @Override
    public void previousPaymentInfo(){
         throw new UnsupportedOperationException();
    }
    @Override
    public void newPayment(){
         System.out.println("Processing "+name+"'s current payment
                            request.");
         // Some other code, if any
    }
}
```

Inside the `main()` method you create a guest user instance now and try to use your helper class in the same manner. Here is the new client code (notice the changes in bold). For your easy understanding, I added some comments to draw your attention to the code that causes the problem now.

```
class Client {
     public static void main(String[] args) {
          System.out.println("***A demo without LSP.***\n");
        PaymentHelper helper = new PaymentHelper();
        // Instantiating two registered users
        RegisteredUserPayment robinPayment = new
                               RegisteredUserPayment("Robin");
        RegisteredUserPayment jackPayment = new
                               RegisteredUserPayment("Jack");
```

```
        // Adding the users to the helper
        helper.addUser(robinPayment);
        helper.addUser(jackPayment);
        GuestUserPayment guestUser = new GuestUserPayment();
        helper.addUser(guestUser);
        // Processing the payments using
        // the helper class.
        // You can see the problem now.
        helper.showPreviousPayments();
        helper.processNewPayments();
    }
}
```

This time you get a surprise and encounter an exception that may look like the following:

```
***A demo without LSP.***
Retrieving Robin's last payment details.
------
Retrieving Jack's last payment details.
------
Exception in thread "main" java.lang.UnsupportedOperationException
    at jdp3e.solid_principles.without_lsp.GuestUserPayment.previousPaymentInfo(GuestU
    at jdp3e.solid_principles.without_lsp.PaymentHelper.showPreviousPayments(PaymentH
    at jdp3e.solid_principles.without_lsp.Client.main(Client.java:23)
```

You can see that although the `GuestUserPayment` class implements the `Payment` interface, it causes `PaymentHelper` to break. You can understand that the loop

```
public void showPreviousPayments() {
        for (Payment payment: payments) {
            payment.previousPaymentInfo();
            System.out.println("------");
        }
    }
```

causes this trouble. In every iteration, you call the method `previousPaymentInfo()` on the respective `Payment` object and the exception is raised for the `GuestUserPayment` instance. So, you now know how a working solution can fail when you add a new code segment. What is the solution? You will find it in the next section.

Better Program

The first obvious solution that may come into your mind is to introduce an `if-else` chain to verify whether the `Payment` instance is a `GuestUserPayment` or a `RegisteredUserPayment`. However, this is not the best possible solution in the sense that if you have another special type of user, you again verify it inside this `if-else` chain. **Most importantly, you violate the OCP each time you modify an existing class that uses this** `if-else` **chain.** So, let's search for a better solution.

In the upcoming program, you remove the `newPayment()` method from the `Payment` interface. You place this method into another interface called `NewPayment`. As a result, now you have two interfaces with the specific operations. Since all types of users can raise a new payment request, the concrete classes of `RegisteredUserPayment` and `GuestUserPayment` both implement the `NewPayment` interface. But you show the last payment detail for the registered users

only. So, the `RegisteredUser` class implements the `Payment` interface. I always advocate for a proper name. Since `Payment` contains the `previousPaymentInfo()` method , it makes sense to choose a better name, such as `PreviousPayment` instead of `Payment`. So, now you see the following interfaces:

```
interface PreviousPayment {
     void previousPaymentInfo();
}
interface NewPayment {
     void newPayment();
}
```

Adjust these new names in the helper class too. Let's look at the updated program which is shown in the following section.

Demonstration 6

Here is the complete demonstration:

```
// PreviousPayment.java
interface PreviousPayment {
     void previousPaymentInfo();
}
// NewPayment.java
interface NewPayment {
     void newPayment();
}
// RegisteredUserPayment.java
class RegisteredUserPayment implements NewPayment,PreviousPayment {
    String name;
    public RegisteredUserPayment(String userName) {
        this.name = userName;
    }
    @Override
    public void previousPaymentInfo(){
        System.out.println("Retrieving "+ name+"'s last
                           payment details.");
        // Some code,if any
    }
    @Override
    public void newPayment(){
         System.out.println("Processing "+name+"'s current payment
                           request.");
         // Some code, if any
    }
}
// GuestUserPayment.java
class GuestUserPayment implements NewPayment {
     String name;
    public GuestUserPayment() {
        this.name = "guest";
    }
    @Override
    public void newPayment(){
         System.out.println("Processing "+name+"'s current payment
                           request.");
         //Some code, if any
    }
}
// PaymentHelper.java
import java.util.ArrayList;
import java.util.List;
```

```
class PaymentHelper {
List<PreviousPayment> previousPayments = new
                            ArrayList<PreviousPayment>();
List<NewPayment> newPayments = new ArrayList<NewPayment>();
    public void addPreviousPayment(PreviousPayment
                                        previousPayment){
        previousPayments.add(previousPayment);
    }
    public void addNewPayment(NewPayment newPaymentRequest){
        newPayments.add(newPaymentRequest);
    }
    public void showPreviousPayments() {
        for (PreviousPayment payment: previousPayments) {
            payment.previousPaymentInfo();
            System.out.println("------");
        }
    }
    public void processNewPayments() {
          for (NewPayment payment: newPayments) {
             payment.newPayment();
            System.out.println("***********");
        }
    }
}
// Client.java


class Client {
     public static void main(String[] args) {
       System.out.println("***A demo that follows the
                               LSP.***\n");
        PaymentHelper helper = new PaymentHelper();
        // Instantiating two registered users.
        RegisteredUserPayment robin = new
                            RegisteredUserPayment("Robin");
        RegisteredUserPayment jack = new
                            RegisteredUserPayment("Jack");
        // Instantiating a guest user's payment.
        GuestUserPayment guestUser1 = new GuestUserPayment();
        // Consolidating the previous payment's info to
        // the helper.
        helper.addPreviousPayment(robin);
        helper.addPreviousPayment(jack);
        // Consolidating new payment requests to
        // the helper
        helper.addNewPayment(robin);
        helper.addNewPayment(jack);
        helper.addNewPayment(guestUser1);
        // Retrieve all the previous payments
        // of registered users.
        helper.showPreviousPayments();
        // Process all new payment requests
        // from all users.
        helper.processNewPayments();
     }
}
```

Output

Here is the output:

```
***A demo that follows the LSP.***
```

```
Retrieving Robin's last payment details.
------
Retrieving Jack's last payment details.
------
Processing Robin's current payment request.
**********
Processing Jack's current payment request.
**********
Processing guest's current payment request.
**********
```

Analysis

What are the key changes? Notice that in demonstration 5, `showPreviousPayments()` and `processNewPayments()` both process `Payment` instances inside the enhanced `for` loop. But in demonstration 6, inside the `showPreviousPayments()` method, you process `PreviousPayment` instances and inside the `NewPayments()` method , you process `NewPayment` instances. This new structure solves the problem you faced in demonstration 5. You can see that this modified design conforms to the LSP because objects are clearly substitutable and the program works properly.

> **Note** I want you to note a minor point. Here the key focus was on the LSP principle, not anything else. You could easily refactor the client code using some static method. For example, you could introduce a static method to display all the payment requests and use this method whenever you need it. But the participant list is very small in this example. So, I have ignored this activity. The same comment applies to similar code segments in this book.

Interface Segregation Principle

You often see a fat interface that contains many methods. A class that implements the interface may not need all these methods. So why does the interface contain all these methods? One possible answer is to support some of the implementing classes of this interface. This is the area the Interface Segregation Principle focuses on. It suggests that you don't pollute an interface with these unnecessary methods only to support one (or some) of the implementing classes of this interface. The idea is that **a client should not depend on a method that it does not use**. Once you understand this principle, you'll see that you used ISP in the better design following the LSP in demonstration 6. For now, let's consider an example with a full focus on the ISP.

> **Points To Remember**
> Note the following points before you proceed further:
>
> - A client means any class that uses another class (or interface).
> - The word "Interface" of the Interface Segregation Principle is not limited to a Java interface. The same concept applies to any supertype, such as an abstract class or a simple parent class.
> - Many examples across different sources explain the violation of the ISP with an emphasis on throwing an exception such as `UnsupportedOperationException()` in Java. Demonstration 7 also demonstrate such an example. It shows the disadvantages of an approach that does not follow the ISP (and the LSP). You saw earlier that

the LSP can deal with this kind of problem. I mentioned this in the code analysis too.

- The ISP suggests that your class should not depend on interface methods that it does not use. This statement will make sense to you when you go through the following example and remember the previous points.

Initial Program

Assume that you have the interface `Printer` with two methods, `printDocument()` and `sendFax()`. There are several users of this class. For simplicity, let's consider two of them only: `BasicPrinter` and `AdvancedPrinter`. Figure 1-1 shows a simple class diagram for this.

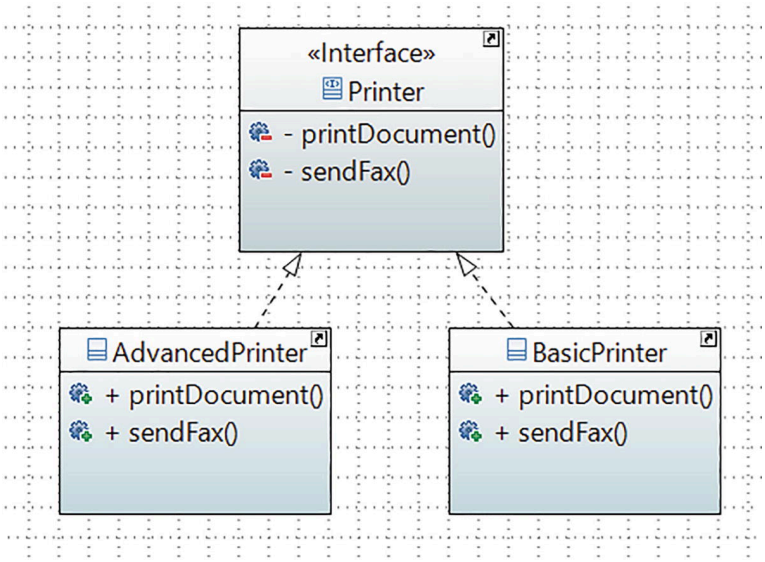

Figure 1-1 The Printer class hierarchy

A basic printer can print documents. It does not support any other functionality. So, `BasicPrinter` needs the `printDocument()` method only. An advanced printer can print documents as well as send faxes. So, the `AdvancedPrinter` needs both methods.

In this case, a change to the `sendFax()` method in `AdvancedPrinter` can force the interface `Printer` to change, which in turn, forces the `BasicPrinter` code to recompile. This situation is unwanted and it can cause potential problems for you in the future.

You saw a problematic situation in Demonstration 5. Later, you saw a solution in demonstration 6 when you segregated the interface `IUser` into `IPreviousPayment` and `INewPayment`. In this case, you followed the ISP too. The ISP suggests you design your interface with the proper methods that a particular client may need.

But **why does a user invite the problem in the first place? Or why does a user need to change a base class (or, an interface)?** To answer this, assume that you want to show which type of fax you are using in a later development phase. You know that there are different variations of fax methods, such as `LanFax`, `InternetFax` (or `EFax`), and `AnalogFax`. So, earlier, the `SendFax()` method did not use any parameters, but now it needs to accept a parameter to show the type of fax it uses.

To demonstrate this further, suppose you have a fax hierarchy that may look like the following:

```
interface Fax {
     void faxType();
}
class LanFax implements Fax {
     @Override
     public void faxType() {
          System.out.println("Using lan fax to send the
                              fax.");
     }
}
class EFax implements Fax {
     @Override
     public void faxType() {
          System.out.println("Using internet fax(efax) to
                              send the fax.");
     }
}
```

To use this inheritance hierarchy, once you modify the `sendFax()` method to `sendFax(Fax faxType)` in the `AdvancedPrinter` class, it demands you change the interface `Printer` (yes, you break the OCP here too). But it is not over yet! When you update `Printer`, you need to update the `BasicPrinter` class too to accommodate this change. **Now you see the problem!**

> **Note** You saw that a change in `AdvancedPrinter` causes changes in the interface `Printer`, which in turn causes `BasicPrinter` to update its fax method. So, you can see that though the `BasicPrinter` does not need this fax method at all, still a change in `AdvancedPrinter` causes it to change and recompile. The ISP suggests you take care of this kind of scenario. This is why when you see a fat interface, ask yourself if these methods are required for a client. If not, split it into smaller interfaces that are relevant to clients.

If you understand the previous discussion, you may not start with the following code where you assume that you may need to support different devices/printers in the future:

```
interface Printer {
     void printDocument();
     void sendFax();
}
```

If you start your coding considering the advanced printers that can print as well as send a fax, it is fine. But in a later stage, if your program needs to support basic printers too, you may write something like

```
class BasicPrinter implements Printer {
     @Override
     public void printDocument() {
          System.out.println("The basic printer prints a document.");
     }
     @Override
     public void sendFax() {
          throw new UnsupportedOperationException();
     }
```

```
}
```

You have already seen that this code can cause a potential problem for you! It is pretty clear that a basic printer does not need to send a fax. But since `BasicPrinter` implements `Printer`, it needs to provide a `sendFax()` implementation. As a result, when `sendFax()` changes in the `Printer` interface, `BasicPrinter` needs to accommodate the change. **The ISP suggests you avoid this kind of situation.**

In this context, can you remember the issue in demonstration 5? When you throw the exception and try to use polymorphic code in an incorrect way, you see the impact of violating the LSP. Once you modify `Printer`, you violate the OCP too.

So, in this case, inside `main()`, you cannot write polymorphic code like the following (because the last line of this code segment will throw the runtime error):

```
Printer printer = new AdvancedPrinter();
printer.printDocument();
printer.sendFax();
printer = new BasicPrinter();
printer.printDocument();
// printer.sendFax(); // Will throw error
```

Also, you cannot write something like

```
List<Printer> printers = new ArrayList<Printer>();
printers.add(new AdvancedPrinter());
printers.add(new BasicPrinter());
for (Printer device : printers) {
  device.printDocument();
  // device.sendFax(); // Will throw error
}
```

In both cases, you will see runtime exceptions.

Point To Note

I want you to note two points, which you probably know. But it is worth mentioning them to avoid any confusion.

First, in this book, I use enhanced `for` loops a lot. But lambda lovers can replace a code segment like

```
for (Printer device : printers) {
    device.printDocument();
}
```

with

```
printers.forEach(Printer::printDocument);
```

You choose which one you want to use.

Second, I showed a small code segment using `List` and `ArrayList`. When you use them in your program, do not forget to import the following import statements:

```
import java.util.List;
```

```
import java.util.ArrayList;
```

Demonstration 7

In previous sections, you started with a program that does not follow a specific principle. Let's repeat the same. Here is the complete demonstration that does not follow the ISP. All parts of the program are inside the package `jdp3e.solid_principles.without_isp`. Refer to this package when you download the source code from the Apress website.

```
// Printer.java
interface Printer {
     void printDocument();
     void sendFax();
}
// BasicPrinter.java
class BasicPrinter implements Printer {
     @Override
     public void printDocument() {
          System.out.println("The basic printer prints a document.");
     }
     @Override
     public void sendFax() {
          throw new UnsupportedOperationException();
     }
}
// AdvancedPrinter.java
class AdvancedPrinter implements Printer {
     @Override
     public void printDocument() {
          System.out.println("The advanced printer prints a document.");
     }
     @Override
     public void sendFax() {
      System.out.println("The advanced printer sends a fax.");
     }
}
// Client.java


class Client {
     public static void main(String[] args) {
          System.out.println("***A demo without ISP.***");
          Printer printer = new AdvancedPrinter();
          printer.printDocument();
          printer.sendFax();
          printer = new BasicPrinter();
          printer.printDocument();
          // printer.sendFax(); // Will throw error
     }
}
```

Output

When you run this program, you will see the following output:

```
***A demo without ISP.***
The advanced printer prints a document.
The advanced printer sends a fax.
The basic printer prints a document.
```

Analysis

You can see that to prevent the runtime exceptions, I needed to comment out a line of code. I kept this dead code for this discussion. You know that you should avoid this kind of commented code because it can cause potential problems in the long run. Since no one touches the commented code, there is a possibility that in the future lots of changes will occur in the codebase and then this code will become irrelevant. So, when a new developer reads the code, they will be clueless about it.

Most importantly, as said before, in this design, if you change the `sendFax()` method signature in `AdvancedPrinter`, you need to adjust the change in `Printer`, which causes `BasicPrinter` to change and recompile. I discussed this in detail.

Think about the problem from another angle. Assume that you need to support another printer that can print, fax, and photocopy. In this case, if you add a photocopying method in the `Printer` interface, both the existing clients, `BasicPrinter` and `AdvancedPrinter`, need to accommodate the change.

Better Program

Let's find a better solution. You understand that there are two different activities: one is to print some documents and the other is to send a fax. So, in the upcoming example, you create two interfaces named `Printer` and `FaxDevice`. `Printer` contains the `printDocument()` method and `FaxDevice` contains the `SendFax()` method. The idea is simple:

- The class that wants print functionality implements the `Printer` interface, and the class that wants fax functionality implements the `FaxDevice` interface.
- If a class wants both functionalities, it implements both interfaces.

> You should not assume that the ISP says an interface should have only one method. In this example, there are two methods in the `Printer` interface and the `BasicPrinter` class needs only one of them. This is why you see the segregated interfaces with a single method only.

Demonstration 8

Here is the complete implementation. All parts of the program are inside the package `jdp3e.solid_principles.isp`. Refer to this package when you download the source code from the Apress website.

```
// Printer.java
interface Printer {
     void printDocument();
}
// FaxDevice.java
interface FaxDevice {
     void sendFax();
}
// BasicPrinter.java
class BasicPrinter implements Printer {
     @Override
     public void printDocument() {
          System.out.println("The basic printer prints a document.");
     }
```

```
}
// AdvancedPrinter.java
class AdvancedPrinter implements Printer,FaxDevice {
     @Override
     public void printDocument() {
          System.out.println("The advanced printer prints a document.");
     }
     @Override
     public void sendFax() {
          System.out.println("The advanced printer sends a fax.");
     }
}
// Client.java
class Client {
     public static void main(String[] args) {
        System.out.println("***A demo that follows ISP.***");
        Printer printer = new BasicPrinter();
        printer.printDocument();
        printer = new AdvancedPrinter();
        printer.printDocument();
        FaxDevice faxDevice = new AdvancedPrinter();
        faxDevice.sendFax();
     }
}
```

Output

Run this program. You will see the following output:

```
***A demo that follows ISP.***
The basic printer prints a document.
The advanced printer prints a document.
The advanced printer sends a fax.
```

Analysis

What happens if you use a default method inside the interface? Let me tell you.

- The foremost point is, prior to Java 8, interfaces couldn't have default methods. All those methods were abstract by default.
- Secondly, if you use a default method inside the interface or an abstract class, I remind you that each time you add a method in the base class (or interface), the method is available for use in the derived classes. This kind of practice can violate the OCP and the LSP, which in turn causes difficult maintenance and reusability issues. For example, if you provide a default fax method in an interface (or an abstract class), the `BasicPrinter` must override it, saying something similar to the following:

```
@Override
public void sendFax() {
   throw new UnsupportedOperationException();
}
```

and you saw the potential problem with this!

- But what happens if you use an empty method, instead of throwing the exception? Yeah, the code will work, but for me, providing an empty method for a feature that is not supported at all is not a good solution in a case like this. From my point of view, it is misleading because the clients see no change in output when they invoke a valid method.

> There is an alternative technique to implementing the ISP. This can be done using the "delegation" technique. The discussion of this is beyond the scope of this book. But remember the following point: delegations increase the runtime (it can be small, but it is nonzero for sure) of an application, which can affect the performance of the application. Also, based on a particular design, a delegated call can create some additional objects. Unnecessary creation of objects can cause trouble for you because they occupy memory blocks. So, to make an application that should work properly using a very small amount of memory (such as a real-time embedded system), you need to be careful before you create an object.

Dependency Inversion Principle

The DIP covers two important things:

- A high-level concrete class should not depend on a low-level concrete class. Instead, both should depend on abstractions.
- Abstractions should not depend upon details. Instead, the details should depend upon abstractions.

You'll examine both points.

The reason for the first point is simple. If the low-level class changes, the high-level class needs to adjust to the change; otherwise, the application breaks. What does this mean? It means you should avoid creating a concrete low-level class inside a high-level class. Instead, you should use abstract classes or interfaces. As a result, you remove the tight coupling between the classes.

The second point is also easy to understand when you analyze the case study discussed in the ISP section. You saw that if an interface needs to change to support one of its clients, other clients can be impacted due to the change. No client likes to see such an application.

So, in your application, if your high-level modules are independent of low-level modules, you can reuse them easily. This idea also helps you design nice frameworks.

> In his book *Agile Principles, Patterns and Practices in C#,* Robert C. Martin explains that a traditional software development model in those days (such as structured analysis and design) tends to create software where high-level modules used to depend on low-level modules. But in OOP, a well-designed program opposes this idea. It inverts the dependency structure that often results from a traditional procedural method. This is the reason he used the word "inversion" in this principle.

Initial Program

Assume that you have a two-layer application. Using this application, a user can save an employee ID in a database. To demonstrate this, let's use

a console application instead of a GUI application. You have two classes, `UserInterface` and `OracleDatabase`. As per its name, `UserInterface` represents a user interface such as a form where a user can type an employee ID and click the Save button to save the ID in a database. `OracleDatabase` is used to mimic an Oracle database. Again, for simplicity, there is no actual database in this application and there is no code to validate an employee ID. Here your focus is on the DIP only, so those discussions are not important.

By using the `saveEmployeeId()` method of `UserInterface`, you can save an employee id to a database. Notice the code for the `UserInterface` class:

```
class UserInterface {
     private OracleDatabase oracleDatabase;
     public UserInterface() {
          this.oracleDatabase = new OracleDatabase();
     }
     public void saveEmployeeId(String empId) {
          // Assuming that this is a valid data.
          // So, storing it in the database.
          oracleDatabase.saveEmpIdInDatabase(empId);
     }
}
```

You instantiate an `OracleDatabase` object inside the `UserInterface` constructor. Later you use this object to invoke the `saveEmpIdInDatabase()` method, which does the actual saving inside the Oracle database. This style of coding is very common. But there are some problems. I'll discuss them in the analysis section before we look at a better approach. For now, see the complete program, which does not follow the DIP.

Demonstration 9

Here is the complete demonstration. All parts of the program are inside the package `jdp3e.solid_principles.without_dip`. Refer to this package when you download the source code from the Apress website.

```
// UserInterface.java
class UserInterface {
     private OracleDatabase oracleDatabase;
     public UserInterface() {
          this.oracleDatabase = new OracleDatabase();
     }
     public void saveEmployeeId(String empId) {
          // Assuming that this is a valid data.
          // So, storing it in the database.
          oracleDatabase.saveEmpIdInDatabase(empId);
     }
}
// OracleDatabase.java
class OracleDatabase {
       public void saveEmpIdInDatabase(String empId){
       System.out.println("The id: "+empId+" is saved in the
                          Oracle database.");
    }
}
// Client.java
class Client {
     public static void main(String[] args) {
```

```
            System.out.println("***A demo without DIP.***");
            UserInterface userInterface = new UserInterface();
            userInterface.saveEmployeeId("E001");
      }
}
```

Output

Here is the output:

```
***A demo without DIP.***
The id: E001 is saved in the Oracle database.
```

Analysis

The program is simple, but it suffers from the following issues:

- The top-level class (`UserInterface`) has too much dependency on the bottom-level class (`OracleDatabase`). These two classes are tightly coupled. So, in the future, if the `OracleDatabase` class changes (for example, when you change the signature of the `saveEmpIdInDatabase(...)` method), you need to adjust the `UserInterface` class.
- The low-level class should be available before you write the top-level class. So, you are forced to complete the low-level class before you write or test the high-level class.
- What if you use a different database? For example, you may want to switch from the Oracle database to a MySQL database or you may need to support both.

Better Program

In this program, you see the following hierarchy:

```
// Database.java
interface Database {
     void saveEmpIdInDatabase(String empId);
}
// OracleDatabase.java
class OracleDatabase implements Database {
     @Override
     public void saveEmpIdInDatabase(String empId) {
          System.out.println("The id: " + empId + " is saved
                                in the Oracle database.");
     }
}
```

The first part of the DIP suggests that we focus on abstraction. This makes the program efficient. So, this time the `UserInterface` class targets the abstraction `Database`, instead of a concrete implementation such as `OracleDatabase`. Here is the new construct of `UserInterface` :

```
class UserInterface {
  Database database;
  public UserInterface(Database database) {
   this.database = database;
  }
```

```
  public void saveEmployeeId(String empId) {
   database.saveEmpIdInDatabase(empId);
  }
}
```

This gives you the flexibility to consider a new database, such as `MYSQLDatabase` as well. Figure 1-2 describes the scenario.

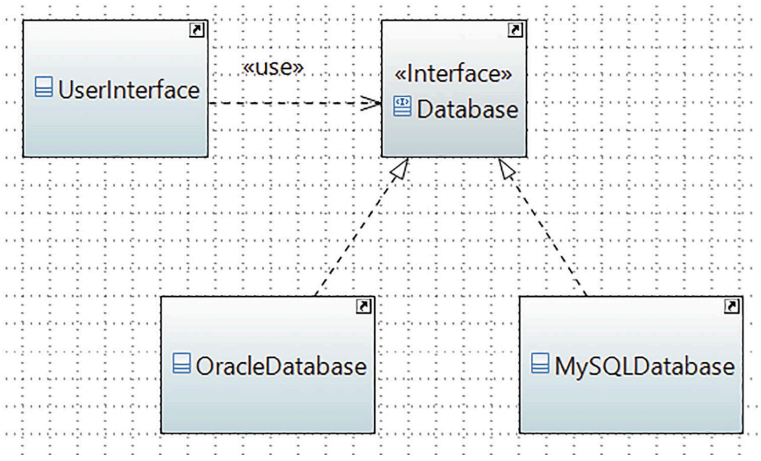

Figure 1-2 The high-level class UserInterface depends on the abstraction Database. The concrete classes OracleDatabase and MySQLDatabase also depend on Database

The second part of the DIP suggests making the `Database` interface considering the need for the `UserInterface` class. It is important because if an interface needs to change to support one of its clients, other clients can be impacted due to the change.

Demonstration 10

All parts of the program are inside the package `jdp3e.solid_principles.dip`. Refer to this package when you download the source code from the Apress website. Here is the complete program for you:

```
// UserInterface.java
class UserInterface {
     Database database;
     public UserInterface(Database database) {
          this.database = database;
     }
     public void saveEmployeeId(String empId) {
          database.saveEmpIdInDatabase(empId);
     }
}
// Database.java
interface Database {
     void saveEmpIdInDatabase(String empId);
}
// OracleDatabase.java
class OracleDatabase implements Database {
     @Override
     public void saveEmpIdInDatabase(String empId) {
          System.out.println("The id: " + empId + " is saved
                                  in the Oracle database.");
     }
}
// MySQLDatabase.java
class MySQLDatabase implements Database {
     @Override
     public void saveEmpIdInDatabase(String empId) {
```

```
          System.out.println("The id: " + empId + " is saved
                              in the MySQL database.");
     }
}
// Client.java
class Client {
     public static void main(String[] args) {
          System.out.println("***A demo that follows the DIP.***");
          // Using Oracle now
          Database database = new OracleDatabase();
          UserInterface userInterface = new
                                    UserInterface(database);
          userInterface.saveEmployeeId("E001");
          // Using MySQL now
          database = new MySQLDatabase();
          userInterface = new UserInterface(database);
          userInterface.saveEmployeeId("E002");
     }
}
```

Output

Here is the output that you'll see when you run this program:

```
***A demo that follows the DIP.***
The id: E001 is saved in the Oracle database.
The id: E002 is saved in the MySQL database.
```

Analysis

You can see that this program resolves all the potential issues of the program in demonstration 9. In short, in OOP, I suggest following the Robert C. Martin quote:

> *High-level modules simply should not depend on low-level modules in any way.*

So, when you have a base class and a derived class, your base class should not know about any of its derived classes. But there are few exceptions to this suggestion. For example, consider the case when your base class needs to restrict the count of the derived class instances at a certain point.

One last point. You can see that in demonstration 10, the `UserInterface` class constructor accepts a `database` parameter. You can provide an additional facility to a user when you use both the constructor and the setter method (`setDatabase`) inside this class. Here is sample code for you. Notice the additional code in bold.

```
class UserInterface {
     Database database;
     public UserInterface(Database database) {
          this.database = database;
     }
     public void setDatabase(Database database) {
          this.database = database;
     }
     public void saveEmployeeId(String empId) {
          database.saveEmpIdInDatabase(empId);
     }
}
```

What is the benefit? Now you can instantiate a database while instantiating the `UserInterface` class and change the target database later using the setter method. Here is sample code. You can append the last part of this segment at the end of `main()` to see new output.

```
// Using MySQL now
database = new MySQLDatabase();
userInterface = new UserInterface(database);
userInterface.saveEmployeeId("E002");
// Changing the target database
userInterface.setDatabase(new OracleDatabase());
userInterface.saveEmployeeId("E002");
```

The previous segment can produce the following output:

```
The id: E002 is saved in the MySQL database.
The id: E002 is saved in the Oracle database.
```

You can follow the same technique for similar examples that are used in this book.

Summary

The SOLID principles are the fundamental guidelines in object-oriented design. They are high-level concepts that help you develop better software. They are neither rules nor laws, but they help you think of possible scenarios/outcomes in advance. I recommend that you understand these principles well before you start reading the design patterns in the following chapters.

In this chapter, I showed you applications that follow (and do not follow) these principles and discussed the pros and cons. Let's have a quick review.

The SRP says that **a class should have only one reason to change.** Using the SRP means you can write cleaner and less fragile code. You identify the responsibilities and make classes based on each responsibility. What is a responsibility? It is a reason for a change. But you should not assume that a class should have a single method only. If multiple methods help you achieve a single responsibility, your class can contain all of these methods. It's ok to bend this rule based on the nature of possible changes. The reason for this is if you have too many classes in an application, it is difficult to maintain. The idea is when you know this principle and think carefully before you implement a design, you can avoid the typical mistakes discussed earlier.

Robert C. Martin mentioned the OCP as the most important object-oriented design principle. The OCP suggests that **software entities (a class, module, method, etc.) should be open for extension, but closed for modification.** The idea is if you do not touch running code, you do not break it. For the new features, you add new code but do not disturb the existing code. It helps you save time on retesting the entire workflow again; instead, you focus on the newly added code and test that part. This principle is often hard to achieve, but partial OCP compliance can provide a bigger benefit to you in the long term. In many cases, when you violate the OCP, you break the SRP too.

The idea of the LSP is **you should be able to substitute a parent (or base) type with a subtype**. It is your responsibility to write true polymorphic code using the LSP. This principle is very important. We discussed it with a typical case study. Using this principle, you can avoid the long tail of `if-else` chains and make your code OCP compliant too.

The idea behind the ISP is that **a client should not depend on a method that it does not use**. This is why you may need to split a fat interface into multiple interfaces to make a better solution. I showed you a simple technique to implement the idea. When you do not modify an existing interface (or an abstract class or a simple base class), you follow the OCP. You have seen that throwing an exception, such as `UnsupportedOperationException()`, can break the LSP. This is why an ISP-compliant application can help you to make OCP- and LSP-compliant applications too. You can make an ISP-compliant application using the delegation technique, which I did not discuss in this book. But the important point is when you use the delegation, you increase the runtime (you may say it is negligible, but it is non-zero for sure), which can affect a time-sensitive application. Using delegation, you may create a new object when a client uses the application. This may cause memory issues in certain scenarios.

The DIP suggests two important points. **First, a high-level concrete class should not depend on a low-level concrete class. Instead, both should depend on abstractions. Second, the abstractions should not depend upon details. Instead, the details should depend upon abstractions**. When you follow the first part of the suggestion, your application is efficient and flexible; you can consider new concrete implementations in your application. When you analyze the second part of this principle, you see that you should not change an existing base class or interface to support one of its clients. This can cause another client to break, and you violate the OCP in such a case. You explored the importance of all these points with examples.

Previous chapter
< Part I. Foundation

Next chapter
2. Simple Factory Pattern >